

Multi-threading optimization for TSP

Using lock-free programming to improve the speed of branch and bound, on the Traveling Salesman Problem.

PCM Course - 2025

Olivia Manz and Samuel Roland

Table of contents

Introduction	2
Data structures	2
TSPParraTask & ParallelTaskRunner	2
Stop management	2
PriorityStack	3
LockFreeQueue	3
Work-stealing deque	4
Memory ordering challenges	5
Integration	6
Memory model	7
Crashes	7
Measures	8
Benchmarking system	8
Results	10
Baseline comparisons results	10
Speedup results	10
Efficiency	11
Conclusion	12
Bibliography	13

Introduction

The goal of this project is to optimize the repartition of work among a large numbers of threads. The TSP has been chosen for this challenge, with a starting code implementing an existing single-threaded resolution. We had access to a 256 cores and 126GB of RAM machine remotely to test our program. The core of the work is on the internal lock-free data structures, that enable an optimized work repartition and synchronisation.

The main ideas were to implement a lock free queue to replace the existing stack. This implementation also required to migrate part of the code to be safe in a multithreaded context. To further optimize the first results, we search for dedicated structures for this kind of problem and found the work-stealing deque, with the paper “Dynamic circular work-stealing deque” [1]. We measured the speedup and efficiency of the final version. Along the way we measured several combinations of the 3 available parameters: number of cities, number of thread and cutoff.

Data structures

In this section we present the structure of 2 core datastructures and one secondary structure also used. The first part is developed inside `parrallel_work.hpp`.

TSPParraTask & ParallelTaskRunner

Stop management

To manage the exit of threads properly, all threads are checking a global counter to detect the end of the problem. For this, we initialize a global `std::atomic<uint64_t> _tasks_done;` to zero and a constant counter `uint64_t _total_todo_tasks_counter;` to the total amount of tasks to cover. To calculate how much tasks a tree or subtree contains, we precalculated at compile-time the amount of nodes in any tree for 0 to 25 cities in a global constant

`SUBTREE_NODES_COUNT_BY_TREE_HEIGHT`. As an example, the tree with 3 cities is composed of 5 nodes (level 0: 1 node, level 1: 2 nodes, level 2: 2 nodes. The sum is $1 + 2 + 2 = 5$.) This allow us to know how many nodes (or tasks) there is for a path with a given number of city. When the cutoff of .i.e 8 is reached, we can cut a subtree of all subpath of 8 cities. In this case, it would increment the counter of `SUBTREE_NODES_COUNT_BY_TREE_HEIGHT[8] = 13700` elements with this call `_tasks_done.fetch_add(13700, std::memory_order_relaxed)`. We can simplify the memory ordering constraints from sequential consistency to `relaxed` because this is related to any other variable that would need to be changed in relation.

We chose to use a `uint64_t` type for the global task counter because the number of tasks grows exponentially with the size of the problem. Consequently, the counter must be able to represent very large values without overflowing. Moreover, since this counter is accessed and updated concurrently by multiple threads, it must be declared as atomic. In practice, atomic operations in C++ are only supported on primitive types, which makes `std::atomic<uint64_t>` a natural and efficient choice.

In addition, we deliberately chose an incrementing counter rather than a decrementing one to reduce the risk of incorrect termination due to underflow. When using an unsigned type, decrementing below zero causes a wrap-around to a very large value, which may result in the stop condition based on comparisons (e.g. > 0) never satisfied. By contrast, an incrementing counter avoids this class of errors: even if the counter were to exceed the theoretical number of tasks, the termination condition would still eventually be met. Although such an overshoot

should not occur in a correct execution, this design choice provides an additional safety margin and helps prevent situations where the program might fail to terminate.

At the end of the problem, only a few remaining threads are working on the last tasks, causing a high amount of threads to try stealing empty queues. To avoid slowing down the end of the execution, we implemented a way for stealers threads to exit if 90% of the tasks have been managed. If they have stolen every available queue without any success and this

`QUIT_THRESHOLD = 0.90;` has been reached, they can stop themselves. In addition to this, we stopped checking the `_tasks_done` counter from the main thread, because this is an active wait. Instead, we just relied on calling `join()` on each thread directly, to be in passive wait.

PriorityStack

This is small structure which inherits `TaskCollection`, used in `ParallelTaskRunner`. This is not thread-safe but it is not shared between threads. It is given to `Task::split()` to store the sub tasks based on a given task.

After performing the split, the thread that generated the subtasks continues its execution by selecting the next task to process from its local collection. It pops a task from the collection, which yields the subtask with the lowest estimated cost, i.e. the most promising candidate for producing a shorter complete tour. By prioritizing such tasks, the algorithm is more likely to discover a high-quality solution early, which in turn tightens the global shortest-distance bound and enables more aggressive pruning of the search space.

To build this priority system, we use a simple `std::vector`, where we just insertion at the right place and pop at the back. The sort criterion is an estimated cost.

The remaining subtasks are pushed into the thread's work-stealing deque, where they become available to other threads. This approach combines local depth-first exploitation of promising branches with global load balancing, ensuring both efficient exploration and effective parallelism.

```
void push(Task* t) override {
    TSPParraTask* tp = static_cast<TSPParraTask*>(t);
    // sorted insert
    auto it = std::upper_bound(_tab.begin(), _tab.end(), tp,
        [](TSPParraTask* a, TSPParraTask* b) { return a->get_estimated_cost() < b->get_estimated_cost(); });
    _tab.insert(it, tp);
}
```

Listing 1:

LockFreeQueue

This implementation is available in file `LockFreeQueue.hpp`. This implementation is based on the course, in chapter 5, with memory ordering constraints added. It works with 2 pointers `head` and `tail` that are `std::atomic`, they are pointing on a dummy node at start. They allow to build a simply-linked list to represent this queue, as a chain of `Node`. Instead of using null pointers to inform about the empty state, we return a boolean value (`false` if empty) and return the `result` by changing the pointer's reference.

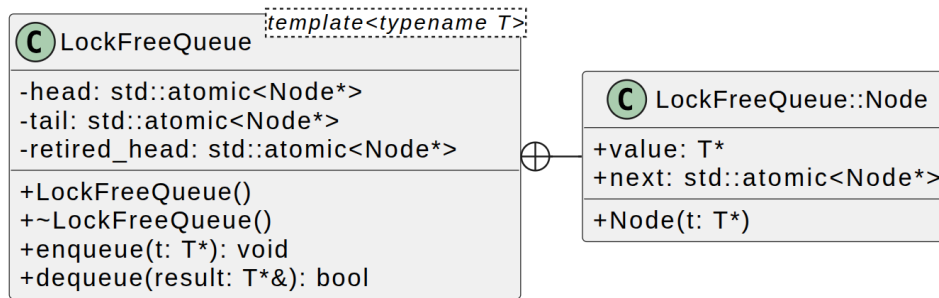


Figure 1: Class diagram of the 2 classes for `LockFreeQueue`

One change from the course's definitions, is the fact that we needed to differ the freeing of removed nodes. We implemented this behavior in `dequeue` with another internal linked list pointed by `retired_head`. Once a node is the result has been read from the first node, we removed it from the queue, we migrate it in "the retire list", which is freed in the destructor.

This queue was initially used to store the list of shared tasks before the task stealing mechanism was implemented. It is now used to manage memory reuse for tasks through a dedicated free list (`_free_list`). We will discuss this in detail in the "Memory Model" chapter.

Work-stealing deque

This part is developed in `wsd.hpp` and integrated in `parallel_work.hpp`.

The structure presented in the paper [1], is a circular and growable buffer, that is used a double ended queue. On one side of the queue (the bottom), the queue's thread owner will be able to push and pop some tasks. At the other end (the top), other threads may attempt to steal tasks, if their own queue is empty.

As visible in Figure 2 `CircularArray` is the internal contiguous structure dynamically allocated, that can double its capacity when the buffer is full. It is using 2 indexes `bottom` and `top`, which are monotonic values (they always increment and never decrement). These indexes are used modulo the size of the buffer to avoid overflows and enable this circular style. Tasks are represented here with a number, to identify the portion of the buffer that is considered to be used.

The `CircularArray` class is used by `CircularWSDeque`.

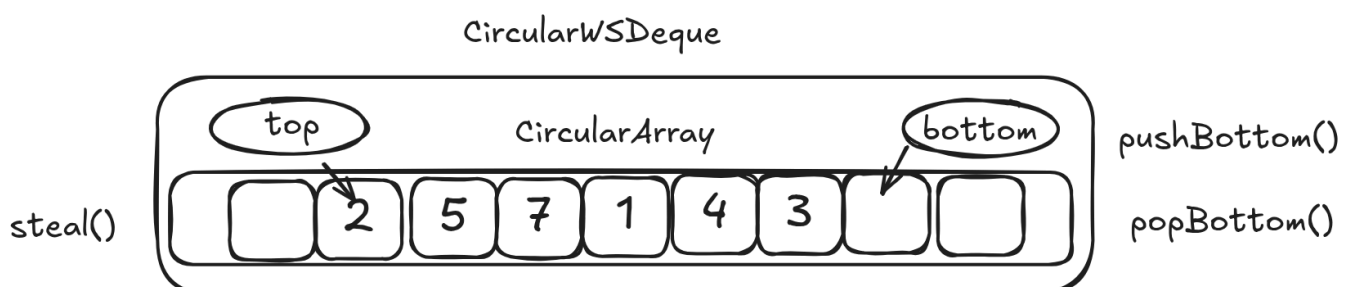


Figure 2: The abstract view of the 2 structures `CircularArray` and `CircularWSDeque`

We are always sure that `top` is lower or equal to `bottom`. `top` is pointing on the top node but `bottom` is pointing on the next cell to be filled by a push on the bottom side. This is leaving an unused cell all the time. The steal on one side `steal()` and the `pushBottom()` and `popBottom()` is done on the other side. This separation helps to reduce the frequency of steal attempts on the owner's work. The `pushBottom` and `popBottom` can only be called by the thread owner of the queue. The `steal` method can be called by any thread. The only concurrency problem we have,

is when the queue is empty or has one element, the `top` could be changed both from `steal` from other threads and by `popBottom`. This is why this index needs to be protected by a CAS (Compare And Swap).

As visible in Figure 3, all attributes that do change are atomic (all of them, except the size and `stored_size` which are constant). Both classes are generic to make it possible to reuse it with other kind of elements outside of the `Task` interface.

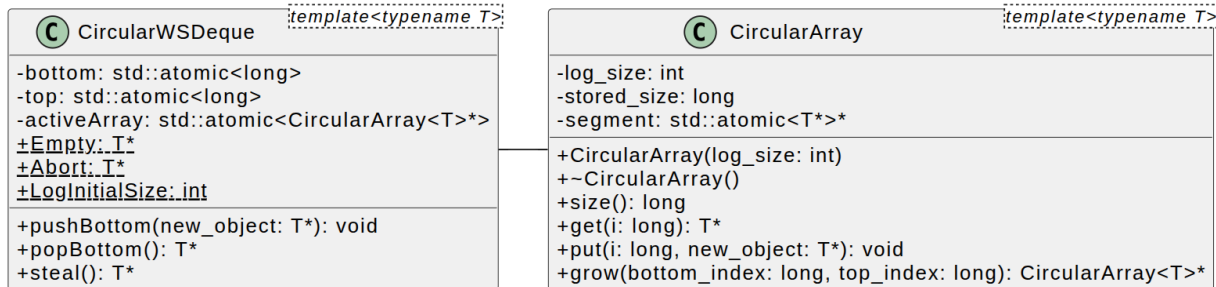


Figure 3: The 2 classes signatures used in the work-stealing implementation

To be able to mark the answer as “empty” (pop and steal) or “abort” (steal), we defined 3 pointers that are markers for such states. To avoid the overhead of a structure with a enum field to discriminate the field, we just defined the empty state as a null pointer. We also defined the abort state as an arbitrary address on the stack, that would never be chosen by a dynamic allocation. With this little hack visible in Listing 2, we can return 8 bytes pointers, instead of 9 or more bytes with a wrapper struct.

```

template<typename T>
class CircularWSDeque {
public:
    static T* Empty;
    static T* Abort;
    // ...
}
static long a = 1;
template<typename T>
T* CircularWSDeque<T>::Empty = nullptr;
template<typename T>
T* CircularWSDeque<T>::Abort = reinterpret_cast<T*>(&a);

```

Listing 2: The initialisation of Empty and Abort static markers pointers in `CircularWSDeque`

The initial size of the array has been defined to 32 in

`int CircularWSDeque<T>::LogInitialSize = 5;` (because 2^5). We didn’t measure the cost of growing the arrays, but this might be something to optimize. This could be also adapted dynamically based on the number of cities.

Memory ordering challenges

One major challenge of this implementation was the memory ordering which gave us some headache. The conversion from the Java code in the paper was not as easy as it seems. First we didn’t had `std::atomic` around `bottom`, which seems to causes issues where it could not warranty the order of operations before its change. As a small example, the code in Listing 3 is a good example where the CPU reordering of instructions could really be bad. If we consider a scenario where the queue is empty, this snippet is pushing a `new_object` but the CPU reorder

the instructions and save `bottom` before the object is really saved in `activeArray` ... This means that an concurrent thread using `steal` on the same queue could take the element without its initialisation to be done.

```
activeArray->put(b, new_object);  
bottom = b + 1;
```

Listing 3: An old code example where `bottom` was not `std::atomic`, inside `pushBottom`

To fix the previous issue, we need to make sure instructions cannot be reordered in the wrong way. This is especially the case with lock-free algorithms, where the precise order is giving us very specific coherence or concurrency warranties! In the fixed Listing 4, we now have an `std::atomic` where we can call `store` with the release order, to make sure no instruction done before the store can be seen by other threads after store effect.

```
activeArray->put(b, new_object);  
bottom.store(b + 1, std::memory_order_release);
```

Listing 4: An extract of the current `pushBottom` code, where `bottom` is `std::atomic` and we use memory ordering mode

Integration

Each thread is assigned its own work-stealing deque. To distribute tasks initially and manage stealing, we implemented a simple strategy. The first thread's deque is initialized with the root task. This thread processes the root task and splits it into subtasks, which are added to its deque. Other threads that have no task available at start will come steal tasks from this first deque, to begin their work

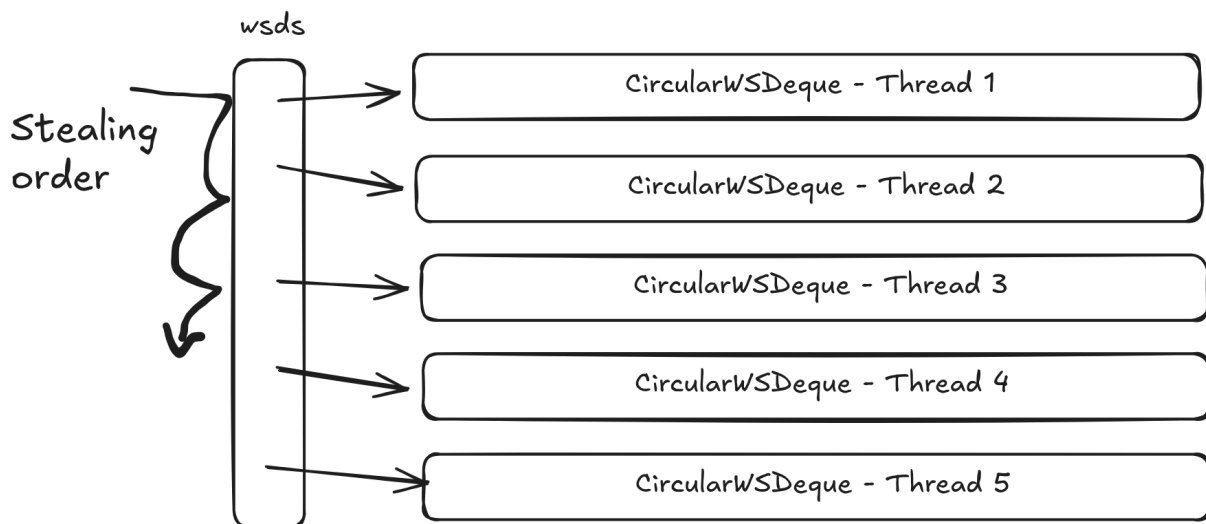


Figure 4: Each thread has it's own work-stealing deque, each thread will start stealing at thread 1 and following

In `ParallelTaskRunner`, the vector of pointer to the deques (`wsds` in the schema) as defined as `std::vector<std::unique_ptr<CircularWSDeque<Task>>> _wsds;`

Potential improvements

This is clearly not the best way to init the deques, as most threads will need to steal thread 1 at the same time, losing some time at start. We didn't have the chance to try it but we would definitely make sure all threads have at least one starting task, to avoid stealing at start. We only

developed one level splitting of the root task, which reduced the total execution time from 16 seconds to 13 seconds in a hand-made test. We think that the ideal solution is to split the root task recursively in breadth, until we have enough tasks to cover all threads. We would push this list one by one to a different deque each time to ensure proper repartition. Then, it would make sense to have the first thread to steal to be the next one (by attributed thread id) and not the first thread, to avoid contention.

Memory model

Our memory management strategy is as following:

- all tasks are dynamically allocated
- all tasks are deallocated only when it has been managed (splitted in subtasks or solved)
- allocation and deallocation are not using `new` and `delete` all the time, a shared pool of heap memory zones is managed (see `_free_list` below)

The `static thread_local LockFreeQueue<Task>* _free_list;` attribute of `TSPParraTask` is used by `reusealloc` and `reusefree`. A shared list (with only `static`, not `thread_local`) had the first advantage of consuming a lower amount of total memory. But it became a central point of contention. This is why we added `thread_local`, so thread has its own list and doesn't wait on other threads. Now that only a single thread is using each list, we could have switched back a non thread-safe stack or queue implementation to avoid CAS operations.

The work-stealing pointer to the circular buffer `activeArray` is not cleaned up after a `grow()` because other stealers might still reference the old pointer. We knowingly leak this memory by not calling `delete`. The paper describes a way to reuse this memory but this didn't seem to be an issue, especially with the 126GB of the server. The leaked amount of memory is almost linear to the number of threads (considering there are an almost fixed number of reallocations of and no shrinking).

Crashes

We had a very hard time debugging and thus we spend a significant amount of time trying to fix numerous kind of crashes (use-after-free, segfault of task pointer, out of range on bitset, pop on empty path...). We still have segfaults or some exceptions throwing for cities ≥ 15 . These issues have significantly slowed us down and made it impossible to measure time for more than 16 cities on the server. By the repetition of the last benchmarks, it seems we were able to fix some of them along the way and we restarted them enough to have some measures to discuss.

```

Thread 247 "tsp" received signal SIGSEGV, Segmentation fault.
(gdb) bt
#0  TSPParraTask::split (this=<optimized out>, collection=<optimized out>) at
parallel_work.hpp:270
#1  TSPParraTask::split (this=0x7ffe500ble40, collection=0x7ffdf9779dc0) at
parallel_work.hpp:262
#2  0x000000000040287d in ParallelTaskRunner::recurse
(this=this@entry=0x7fffffffd0b0, t=0x7ffe500ble40, tid=tid@entry=245)
    at parallel_work.hpp:364
#3  0x0000000000402d4d in ParallelTaskRunner::worker (this=0x7fffffffd0b0, tid=245)
    at parallel_work.hpp:434
#4  0x00007ffff7c4e3e4 in execute_native_thread_routine () from /lib64/libstdc+
+.so.6
#5  0x00007ffff7a53464 in start_thread () from /lib64/libc.so.6
#6  0x00007ffff7ad65ac in __clone3 () from /lib64/libc.so.6

```

Listing 5: Example of GDB backtrace, which is not very helpful in itself

Some issues are caused by duplication of tasks. A task must be only managed by one thread, and because of issues with the Work-stealing deque implementation, some tasks are stolen or stolen+popBottom twice or even three times. Asking ChatGPT to write GoogleTest for us (see `wsd_tests.cpp`) has confirmed we have stolen and duplicated tasks. But we could not figure out how to fix our structures. Some issues might be related to memory ordering again.

Measures

Our program has the following arguments.

```

> ./tsp
Usage: ./tsp <file.tsp> [nb cities] [nb threads] [cutoff]

```

We run most of our benchmarks with a Python CLI `bench.py` as a wrapper of `hyperfine`, to make sure small we can take an average of several runs. In practice, this is taking too much time with > 15 cities, so we reduce the maximum of executions count. Hyperfine is choosing itself how much runs to do, but sometimes is too slow to wait for 900 executions, when a lot of different combinations are tested. From 8 cities to 17 and above, we tried to reduce progressively (300 runs for 1-7, 200 runs for 8-9-10-11, 40 runs for 13, 5 runs for 15, 2 runs for 16...). Finally it was too slow, we reduced it to 1 run for cities >= 13.

```

> hyperfine './tsp dj38.tsp 12 50 4'
Benchmark 1: ./tsp dj38.tsp 12 50 4
  Time (mean ± σ):      745.4 ms ±  77.3 ms
  Range (min ... max):  640.8 ms ... 861.4 ms 10 runs

```

Benchmarking system

```

> uv run bench.py init
Machine ID not found, please enter an ID for your machine: srv2
Saved 'srv2' as Machine ID.
No configuration found. Let's set it up.

Enter cities counters (like 5,10,15): 5,10,13

```


Enter threads counters: 256
Enter cutoff values: 0,1,2,3,4,5,6,7,8,9,10,11,12,13

Configuration saved to bench/configs/srv2.json

This is an example of the configuration file generated.

```
{
  "machine_id": "srv2",
  "cities": [ 5, 10, 13 ],
  "threads": [ 256 ],
  "cutoff": [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 ]
}
```

Our system `bench.py` allowed us to compare different versions of our program and compare between execution with the previous results. More features are described in `bench/README.md` if needed. In this report, we have several baselines (several versions) of the code, where we saved the name and Git hash. Here are the baselines we kept for comparisons in this report. Specific versions can be lookup in our GitHub repository.

Name, Commit Hash	Description
direct N/A	The provided code with the direct approach using a single thread
base 3585e48d9d0d325d56fde899f134f84440e9ea3f	First working parallel version using a LockFreeQueue to share a queue of tasks globally
first-wsd 59cc1958518943d4d06f78bea5edb44ed1ce632c	First WorkStealing Deque implementation with a basic stealing strategy and initialisation
final 675509761b81b93639e7820b110e928ef118bf68	Final version with optimisations and fixes around the work-stealing deque

We first tried measuring what is the best cutoff on the server from the `base` version. We created a plot on Figure 5 that shows that the best default cutoff is around 8. This is what we used for our measures, with a pinned value to simplify comparisons.

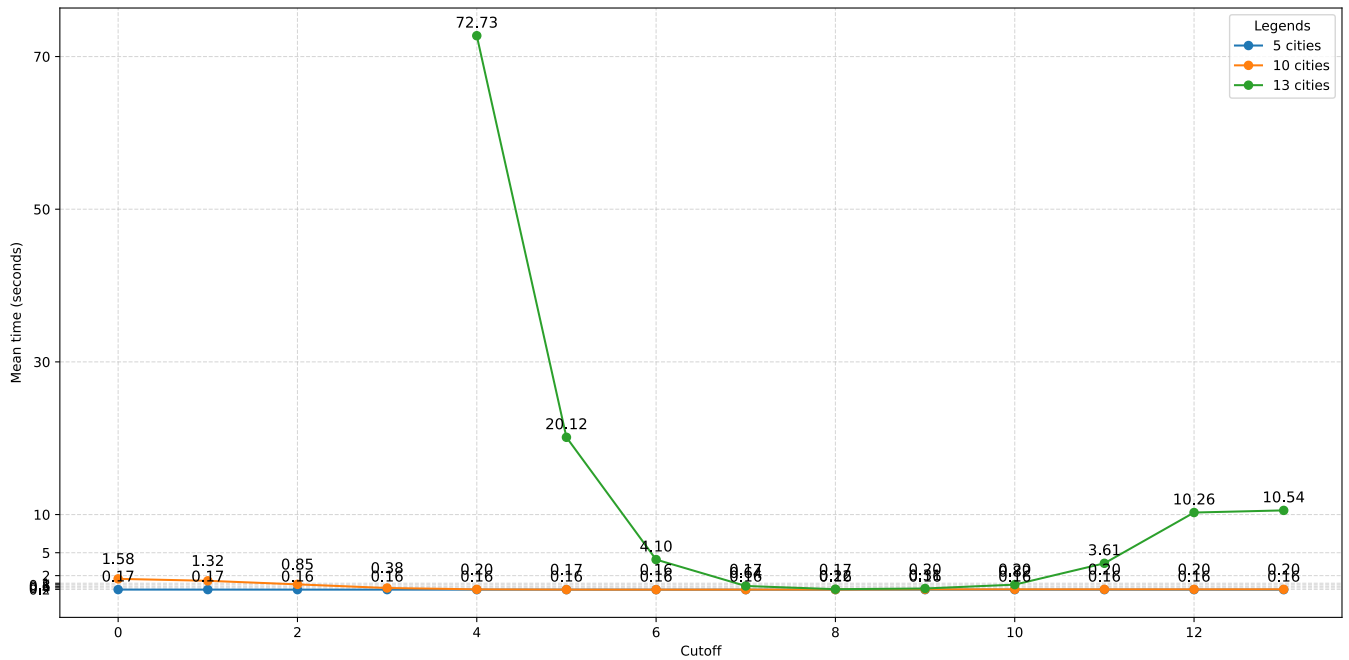


Figure 5: Cutoff analysis - 256 threads - on server

Results

Baseline comparisons results

Along the development of each versions, it was interesting to compare them for a few number of cities to see the progression.

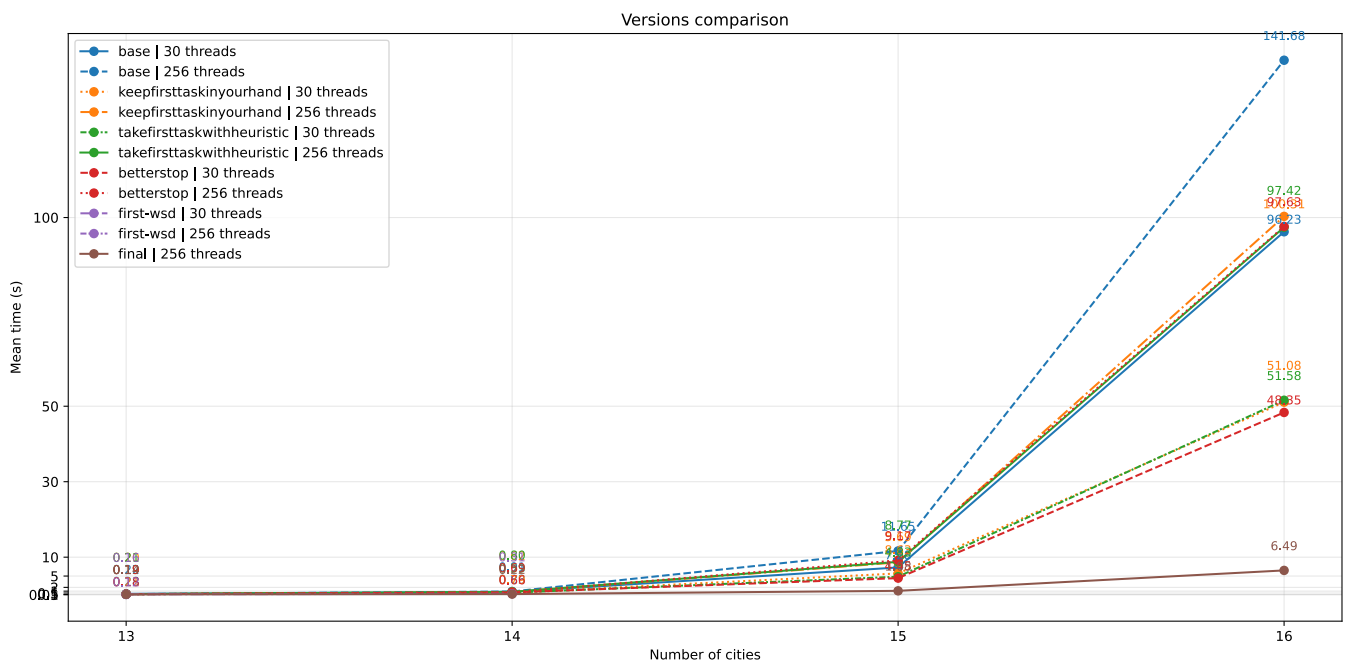


Figure 6: Performance evolution during the development, on the server

Speedup results

Note: to avoid measuring ourself like all other groups the same values for `direct`, we took these specific values from Rafael and Gaspard. We made all the other measures ourself.

This speedup graph of the version `final` compared to the `direct` versions on Figure 7 is showing us a maximum speedup to 45x for 15 cities and 36x for 16 cities. This is strange to see a

higher speedup for 15 cities. The effect of the parallelism has worked as our speedup is only growing with more threads until the number of cores.

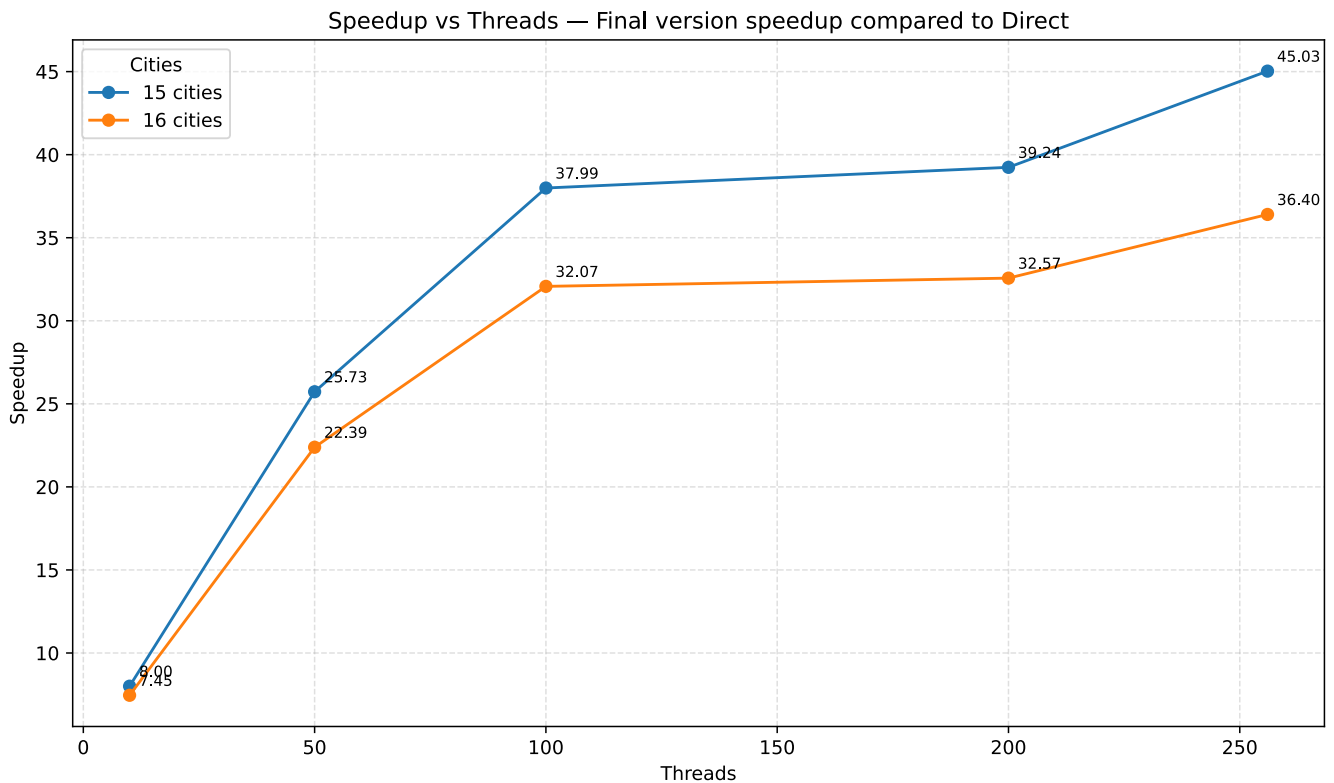


Figure 7: final version speedup compared to direct

Efficiency

As discussed in class, we measured the efficiency of the final version, on several threads count for cities 15 and 16. We are showing the number of abortions (times where a steal operation has aborted because of concurrency) and number of empties (when steal or popBottom is returning empty).

Cities	Threads	Cutoff	Mean time (s)	Aborts	Empties
15	10	8	5.47415	0	265
15	50	8	1.70257	29	6472
15	100	8	1.15291	89	9733
15	200	8	1.11635	162	39265
15	256	8	0.972745	333	121669
16	10	8	28.5893	1	280
16	50	8	9.51987	38	5600
16	100	8	6.64518	218	20800
16	200	8	6.54397	387	34833
16	256	8	5.85474	914	124653

We also visualized this table with dedicated plots Figure 8. We can observe that the more we have threads, the more we are going to lose time in concurrent operations on the same dequeues. This is also caused by the fact that threads are stealing on the same place on the current implementation at start of the execution.

Regarding the empties counter, we see that after 200 threads, the counter is growing faster. This means there a lot of empty queues, which means a lot of threads are inactive (not working on tasks). They are inactive and spend time trying to steal tasks on empty queues most of the time.

This show an imbalance starting at 200 threads of work distribution, some threads are more busy with bigger tasks, a lot of others are waiting and working on small tasks that are quickly solved.

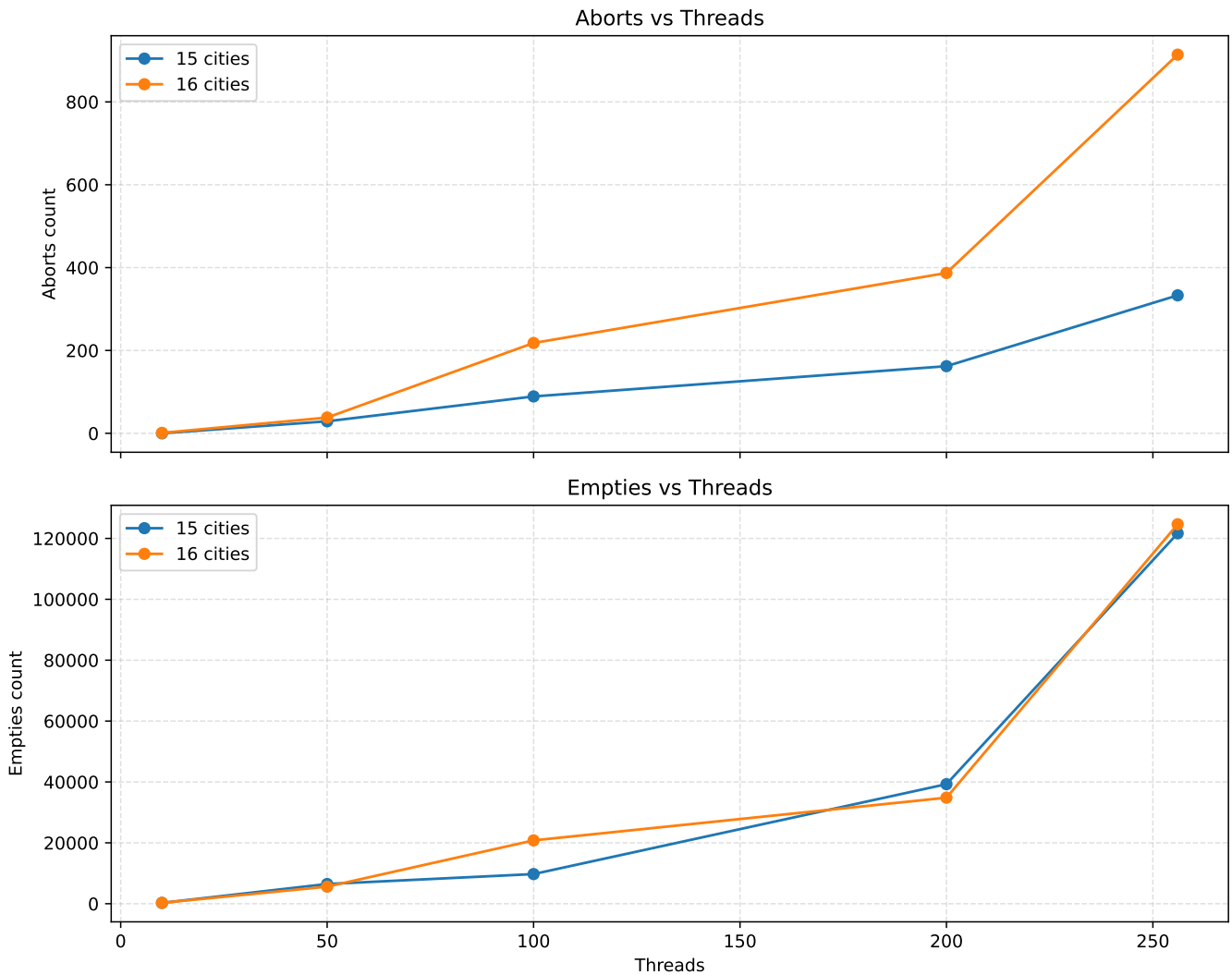


Figure 8: Aborts and empties counters comparison versus threads

It's normal to see the 2 curves to be near, because thread efficiency doesn't depend on the number of tasks (linked to the number of cities), but depends more on the number of threads (because more threads = more dequeues and more activity).

Conclusion

We managed to achieve a maximum of 45X speedup in best cases, between 30x and 40x in average. Our program has still segfaults, but 16 cities are still supported.

We already a few possible improvements for future projects, but here is a brief resume of the priority features

- Fix the concurrency bugs still present on the work-stealing deque. The help of unit tests could be a good way to fix some issues.

- Optimize the initialisation of deques to have at least one task per queue, if not more. Even with a minor change of splitting the root task, this has shown some real potential.
- Clean the code, especially removing the LockFreeQueue usage, which is no longer required in the current version of the code. (because of the `thread_local`). This is probably causing memory overhead to have barriers for a single thread.

Bibliography

- [1] D. C. et al, "Dynamic circular work-stealing deque." Accessed: Jan. 05, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/1073970.1073974>